

18-ai-prompt-engineering

Modul 18: AI Prompt Engineering untuk Developer RPL

Level: □ Intermediate

Jam: 4 (2 sesi × 2 jam)

Prasyarat: Pengalaman coding di bahasa apapun (PHP/JS/TS)

Output: Prompt library + Mastra prompt template

Target: SMK RPL — siswa yang udah pakai ChatGPT buat nyontek coding.

Filosofi: Bukan anti-AI. Tapi AI itu alat, bukan pengganti otak. Lo yang koding, AI yang bantu. Bukan sebaliknya.

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Bedain teknik prompt zero-shot, few-shot, chain-of-thought, role prompting
 - Nulis prompt efektif untuk coding, debugging, dan dokumentasi
 - Pake structured output (JSON mode) buat hasil konsisten
 - Improve prompt secara iteratif
 - Generate kode, test, dan dokumentasi dengan AI
 - Implement Mastra prompt templates di project
 - Kelola versi prompt dan terapkan etika penggunaan AI
-

Materi

Sesi	Topik
1	Prompt Techniques — zero-shot, few-shot, chain-of-thought, role prompting, structured ou
2	Prompt for Code & Production — code gen, unit test, API doc, error explain, code review, l

Output Akhir Modul

Prompt Library + Mastra Prompt Template — kumpulan prompt reusable dan template Mastra untuk berbagai skenario coding.

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- "Bandingin output zero-shot vs few-shot untuk task yang sama"
- "Refine prompt sampe hasilnya sesuai"
- "Generate test case dari deskripsi fungsi"
- "Bikin prompt template Mastra untuk code review"

1. Kenapa AI Mentah Berbahaya

Realita: lo buka ChatGPT, ketik "buatin website crud php", copy paste, lolos tugas. Sekarang lo lulus SMK — lo nggak bisa ngapa-ngapain sendiri. Selamat datang di era kompetisi ketat, lo punya 0 skill.

Masalah besarnya:

- **Lo nggak belajar debugging.** Pas error muncul di production, ChatGPT nggak bisa lo panggil.
- **Lo nggak paham arsitektur.** Lo tahu kode jalan — tapi kenapa jalan, berapa biaya server, gimana scaling-nya? Nol.
- **Plagiarisme itu gampang terdeteksi.** Guru/penguji udah paham pola AI. Kode lo bakal kecium dari 1 kilometer.
- **Kode AI sering ngaco.** Hallucination, security hole, dependency usang. Lo nggak bisa bedain mana yang aman dan mana yang bom waktu.

Aturan emas: Jangan pernah pake kode dari AI yang lo nggak paham setiap barisnya. Kalau lo masih bingung, lo suruh AI jelasin dulu sampe lo ngerti. Baru lo merge kode itu.

Ini bukan tentang "jangan pake AI". Ini tentang **jadi developer yang pake AI, bukan kontraktor yang nge-prompt doang.**

2. Prompt Patterns untuk Coding

Ada 6 pola prompt utama yang lo butuhin sebagai developer:

a. Generate Boilerplate

Prompt yang bikin kerangka kode — CRUD, handler route, setup project.

Buatkan Express.js route handler untuk REST API products. Punya method: GET / (list), GET /:id (detail), POST / (create), PUT /:id (update), DELETE /:id (delete). Gunakan Prisma ORM. Respons JSON standar: { success, data, message }. Jangan kasih implementasi logic bisnis — cukup boilerplate aja.

Kenapa ini aman? Lo minta kerangka, bukan logika. Tugas lo ngisi implementasinya.

b. Refactor

Buat perbaiki kode kotor jadi lebih rapi.

Refactor kode ini:

- Pisahkan logic dari controller ke service layer
- Ganti nested callback jadi async/await
- Tambahin error handling
- Jangan ubah behavior/output

[tempel kode lo]

Syarat: lo harus udah baca kode asli dan hasil refactor. Kalau lo nggak bisa jelasin apa yang berubah, lo belum paham.

c. Debug

Ini yang paling sering disalahgunakan. Cara yang bener:

Saya dapet error ini di [file/baris/konteks]:

[error message / stack trace]

Kode terkait:

[kode yang lo curiga bermasalah]

Yang udah saya coba: [xyz]

Tolong bantu analisis penyebabnya aja — jangan kasih fix langsung.

Saya mau coba fix sendiri dulu.

PENTING: Minta AI jelasin *penyebab*, bukan *solusi*. Lo belajar debugging dengan nyari solusi sendiri. Kalau mentok, baru minta saran.

d. Explain

Ini kunci belajar — minta AI jelasin kode yang lo nggak ngerti.

Jelaskan kode ini baris per baris dengan analogi dunia nyata.

Target: anak SMK RPL semester 3 yang udah paham PHP OOP dasar dan MySQL.

Fokus: kenapa pake pattern ini, bukan apa yang dilakukan.

[kode yang nggak lo paham]

Setelah AI jelasin, lo coba **rephrase pake bahasa lo sendiri** ke AI.

Kalau AI bilang "kurang tepat", ulangi sampe lo beneran paham.

e. Write Tests

Buat unit test untuk fungsi ini pake Jest.

Scope: valid input, invalid input, edge cases.

Jangan ubah kode asli.

[fungsi yang mau dites]

Test yang ditulis AI harus lo **running** dan **paham alurnya**. Kalau ada test gagal — itu kesempatan belajar, bukan error.

f. Generate Dokumentasi

Generate JSDoc untuk kode berikut.

Tipe: function description, param types, return values, contoh usage.

Bahasa: Indonesia.

[kode]

3. Code Review dengan AI

Sebelum commit/gabungkan branch, minta AI review kode lo.

Prompt review:

Review kode ini sebagai senior developer RPL.

Cari:

1. Security vulnerabilities (SQL injection, XSS, CSRF, hardcoded secrets)

2. Performa buruk (N+1 query, loop dalam query)
3. Code smell (duplicate, fungsi kepanjangan, magic number)
4. Best practice violation
5. Maintainability issue

Prioritas: security > performa > readability.
Jangan kasih fix – cukup daftarin masalah dan severity-nya.

[kode lo]

Aturan main: Lo evaluasi tiap temuan AI. Kadang AI salah. Kadang AI ngerekomendasiin refactor yg nggak perlu. Skill lo sebagai developer adalah **memutuskan mana yang diterima**.

4. Context Management

AI cuma sepinter konteks yang lo kasih. Makin bagus konteks, makin bagus output.

Project Context

Kasih AI gambaran proyek lo sebelum minta bantuan:

Project: Aplikasi POS untuk toko kelontong

Stack: Laravel 11, MySQL, Tailwind CSS, Alpine.js

Database: 5 tabel (users, products, categories, transactions, transaction_items)

Saya udah selesai: auth, CRUD produk

Yang lagi dikerjain: laporan transaksi mingguan pake raw SQL

Ini bikin AI ngerti konteks dan nggak ngasih saran irrelevant.

RAG (Retrieval-Augmented Generation)

Kalau lo pake tools seperti Cursor atau Copilot dengan konteks code-base, lo nggak perlu explain ulang — tools ini udah baca kode lo. Tapi tetap aja: makin spesifik prompt lo, makin akurat hasilnya.

Prompt Chains

Jangan nanya semuanya dalam 1 prompt. Breakdown:

PERTANYAAN 1: "Jelasin cara kerja middleware di Laravel"

→ Paham → lanjut

PERTANYAAN 2: "Gimana bikin custom middleware untuk role admin?"

→ Paham → lanjut

PERTANYAAN 3: "Sekarang bantuin implementasi di project saya"
Ini namanya **chain-of-thought prompting**. Lo belajar step by step, bukan loncat ke hasil akhir.

5. Cursor & GitHub Copilot Mastery

Alat utama developer 2025+. Pake bener — lo bisa 2-3x lebih cepat.

GitHub Copilot

Fitur	Shortcut	Guna
Inline suggestion	Tab + Alt+]	Auto-complete kode
Open inline chat	Ctrl+I	Minta bantuan di baris tertentu
Open chat panel	Ctrl+Shift+I	Diskusi general
Fix from AI	/fix	Fix error di terminal/compiler
Explain	/explain	Jelasin selection
Tests	/tests	Generate tests untuk fungsi

Cursor

Fitur	Shortcut	Guna
Chat (model bebas)	Cmd/Ctrl+L	Tanya apa aja
Inline edit	Cmd/Ctrl+K	Edit selection spesifik
Agent mode	Cmd/Ctrl+Shift+A	Eksekusi multi-file task
Terminal AI	Cmd/Ctrl+Shift+L	Debug command

Agent Mode (Cursor): Ini yang paling kenceng tapi paling berbahaya. AI bisa:

- Baca semua file di project
- Edit multiple file sekaligus
- Jalankan command
- Install dependencies

Protokol pake Agent Mode:

1. Selalu commit dulu sebelum prompt Agent (biar gampang roll-back).
2. Minta AI bikin rincian rencana dulu — jangan langsung eksekusi.
3. Review tiap perubahan lewat diff viewer.
4. Kalau ada yg nggak lo paham — tanya "kenapa lo ubah ini?"

Inline Chat vs Full Chat

Inline Chat	Full Chat
Fokus 1 fungsi/blok "Gimana refactor fungsi ini?" Cepet, langsung ke kode	Diskusi arsitektur "Apa beda REST sama GraphQL?" Butuh konteks lebih

Rule of thumb: Kalau yang lo tanya spesifik ke baris kode tertentu → Inline. Kalau konsep umum atau arsitektur → Chat panel.

6. PROMPT-LOG.md Culture

Ini budaya yang harus lo terapkan tiap minggu. File PROMPT-LOG.md ditaro di root project.

Format:

PROMPT-LOG

2025-01-20

Prompt: Generate validasi form registrasi

- **Amanan:** Security (validation rules, XSS prevention)
- **Hasil:** Dapet regex validation + sanitizer
- **Yang dipelajari:** `htmlspecialchars()` dengan flags ENT_QUOTES
- **Modifikasi manual:** Tambahin validasi nomor HP Indonesia

Prompt: Debug login error 500

- **Tujuan:** Cari penyebab error
- **Hasil:** Ternyata ada typo di nama kolom database
- **Yang dipelajari:** Debugging pakai error log Laravel
- **Takeaway:** Selalu cek migration sebelum ngoding

Kenapa ini penting:

1. **Lo bisa tracking** — prompt mana yang efektif, mana yang nggak.
2. **Guru/pembimbing lo bisa lihat** — bukti lo belajar, bukan nyontek.
3. **Bikin lo disiplin** — kalau lo tahu harus nulis log, lo mikir 2x sebelum prompt asal-asalan.
4. **Portofolio belajar** — 6 bulan kemudian lo liat PROMPT-LOG lama, lo bisa ukur progress.

Kebijakan kelas PROMPT-LOG:

- Tiap prompt → minimal 1 kalimat "Apa yang saya pelajari"
 - No log = dianggap plagiarisme
 - Kualitas prompt dinilai (spesifik vs vague)
 - Wajib ada minimal 1 modifikasi manual per prompt
-

7. Pedoman Etis

Garis keras — ini nggak bisa ditawar.

☐ Yang Nggak Boleh

- **Copy paste mentah** tanpa baca → ini nyontek. Polos.
- **Prompt "kerjain tugas gue"** → lo bayar orang lain buat ngerjain. Sama aja.
- **Minta AI nulis esai/tugas teori** → lo nggak belajar apa-apa.
- **Push kode tanpa review** → lo tanggung jawab atas kode yang lo push. Kalau ada security hole gara-gara AI, nama lo yang tercoreng.
- **Pake AI di ujian tanpa izin** — ini udah jelas pelanggaran akademik.

☐ Yang Boleh & Dianjurkan

- **Minta AI jelasin konsep** → belajar 10x lebih cepet.
- **Generate boilerplate** → lo isi logic-nya sendiri.
- **Refactor kode lo sendiri** → terus lo pelajari bedanya.
- **Bantu debugging** — tapi lo tentuin fix-nya.
- **Generate test** → setelah lo paham logic fungsinya.
- **Brainstorming arsitektur** → lo yang putusin akhirnya.

Aturan 70/30

70% kode di project lo harus lo tulis manual. Maksimal 30% dari AI. Kalau lo ngirim project yang 90% AI-generated, lo:

1. Nggak lulus wawancara kerja (mereka tes coding manual)
2. Nggak bisa maintain kode lo sendiri
3. Ngecewain diri lo sendiri

Transparansi

Kalau lo pake AI buat bantu — bilang. Di commit message, di README, di PROMPT-LOG. Transparansi itu bedain profesional dari tukang contek.

8. Workflow Harian Developer + AI

Ini workflow yang lo pake tiap hari:

1. BACA & PAHAMI masalah → tulis di notes sendiri
2. COBA coding manual 15-30 menit → tanpa AI dulu
3. MENTOK? → baru minta AI (explain / saran)
4. TERAPIN solusi manual → dengan pemahaman lo
5. REVIEW & REFACTOR → pake AI buat ngecek kualitas
6. PROMPT-LOG → catet apa yang dipelajari

Ini bukan anti-AI. Ini **pro-AI dengan kendali**. Lo supirnya, AI cuma navigator.

9. Contoh Prompt Baik vs Buruk

❌ Buruk	✅ Baik
"Buatin CRUD user"	"Buat migration + Model User dengan field: name, email, role (enum: admin, user)"
"Fix error ini"	"Error: Call to undefined method. Kode saya: [kode]. Saya udah cek nama methodnya, tolong perbaiki"
"Jelasin OOP"	"Jelasin konsep inheritance di PHP OOP pake analogi warung kopi. Saya udah baca dulu"
"Buatin test"	"Buat unit test pake PHPUnit untuk fungsi hitungDiskon(). Logic: diskon 10% jika total > 10000"

Ciri prompt buruk: vague, nyuruh AI kerja sendiri, nggak ada konteks. **Ciri prompt baik:** spesifik, ada batasan jelas, lo yang kontrol.

10. Assessment

Project akhir modul ini bukan tes teori — tapi **demonstrasi**.

Tugas: AI-Augmented Mini Project

1. Bikin fitur baru di project semester lo (contoh: laporan PDF, export Excel, search filter).
2. Wajib pake AI minimal 3 kali, DIDOKUMENTASIKAN di PROMPT-LOG.md.
3. Di PROMPT-LOG, tiap prompt harus ada:
 - Tujuan prompt
 - Hasil AI (tempel)

- Modifikasi manual yang lo lakukan
 - Pelajaran yang didapat
4. Presentasi: lo jelasin baris per baris dari kode yang AI-generated.
 5. Kalau lo nggak bisa jelasin → lo nggak lulus.

Rubrik Penilaian

Kriteria	Bobot
Kualitas PROMPT-LOG	30%
Pemahaman manual tiap baris kode	40%
Kode berfungsi & aman	20%
Transparansi (ngaku pake AI)	10%

Penutup

AI nggak bakal ilang. Makin canggih. Tapi developer yang cuma bisa nge-prompt — bakal diganti sama developer yang bisa pake AI **plus** paham fundamental.

Lo punya pilihan sekarang:

- **Jalan 1:** Jadi "prompt monkey" — bisa ngomong ke AI, nggak ngerti apa yang terjadi. Gajinya kecil, gampang diganti.
- **Jalan 2:** Jadi developer yang pake AI buat kebut 10x — tapi lo pegang kendali penuh. Lo yang direkrut, bukan AI.

Pilih jalan 2. Start now.

"AI doesn't replace developers who understand code. It replaces those who don't."

Modul 18 — AI Prompt Engineering untuk RPL. Ditujukan untuk pembelajaran, bukan plagiarisme.

Referensi & Tools

- [GitHub Copilot Docs](#)
- [Cursor AI](#) — AI-native editor
- [OpenAI Prompt Engineering Guide](#)

- [Anthropic Prompt Engineering Guide](#)
- [Mastra AI](#) — AI agent framework

1. Prompt Techniques

Durasi: 2 Jam

Fokus: Zero-shot, few-shot, chain-of-thought, role prompting, structured output, iterative refinement

Zero-Shot vs Few-Shot Prompting

Zero-Shot

Zero-shot = kasih instruksi langsung tanpa contoh. AI nebak-context dari training data.

```
// Zero-shot: "Klasifikasikan sentimen"
// Prompt: "Klasifikasikan sentimen dari teks ini: positif/negatif/netral"
// Teks: "Aplikasi ini sering error pas checkout"
```

Masalah: hasil bisa vague atau inconsistent. AI tebak sendiri formatnya.

Few-Shot

Few-shot = kasih 2-5 contoh dulu, baru minta AI kerja.

Aspek	Zero-Shot	Few-Shot
Contoh	0	2-5
Konsistensi	Rendah	Tinggi
Cocok untuk	Task simpel, umum	Task spesifik, butuh format pasti
Panjang prompt	Pendek	Lebih panjang
Akurasi format	Suka melenceng	Stabil

```
// Few-shot: kasih contoh format dulu
const prompt = `
Klasifikasikan sentimen dari teks e-commerce (POSITIF/NEGATIF/NETRAL).`
```

Contoh:

Teks: "Barang sampai cepat, kualitas bagus"

Sentimen: POSITIF

Teks: "Pengiriman telat 3 hari, barang rusak"

Sentimen: NEGATIF

Teks: "Barang sesuai deskripsi"
Sentimen: NETRAL

Sekarang klasifikasikan:
Teks: "Customer service lambat respon, barang OK"
Sentimen:`;

// Output expected: "NEGATIF" – konsisten dengan format contoh

Kapan pake few-shot:

- Format output harus ketat (JSON, enum, list)
 - Task butuh domain knowledge spesifik
 - AI sering salah interpretasi instruksi
-

Chain-of-Thought (CoT)

Chain-of-thought = suruh AI berpikir step-by-step sebelum jawab.

Tanpa CoT (langsung jawab)

*// Prompt: "Hitung diskon: harga 200.000, diskon 15% + tambahan 10rb"
// AI jawab langsung: "20.000" – BISA SALAH*

Dengan CoT

```
const prompt = `  
Hitung total yang harus dibayar.
```

Langkah-langkah:

1. Hitung diskon pertama: 15% dari 200.000 = 30.000
2. Harga setelah diskon: 200.000 - 30.000 = 170.000
3. Potong tambahan 10.000: 170.000 - 10.000 = 160.000

Total: 160.000

```
Sekarang hitung sendiri:  
Harga: 500.000, Diskon: 20% + kupon 25.000  
`;
```

Kenapa CoT penting buat coding:

- Debugging butuh tracing step-by-step

- Refactoring butuh analisis dependensi
- Generate complex query butuh breakdown logika

Zero-Shot CoT

Cukup tambahkan "Mari berpikir step by step" di akhir prompt:

```
// Zero-shot CoT
const prompt = `
Buat function untuk cek apakah suatu string adalah palindrome.
Jelaskan approach lo dulu, baru kasih kode.
Mari berpikir step by step.
`;
```

Role Prompting

Role prompting = kasih AI persona/sistem role biar output sesuai konteks.

System Message Pattern

```
// System prompt – define persona
const systemPrompt = `
Kamu adalah senior software engineer yang specialize di code review.
Prioritas: security > performance > readability.
Gunakan bahasa Indonesia.
Jangan kasih kode fix – cukup analisis dan rekomendasi.
`;

// User prompt – task spesifik
const userPrompt = `
Review kode ini:
function getUser(id) {
  return db.query("SELECT * FROM users WHERE id = " + id);
}
`;
```

Role Comparison

Role	Konteks	Gaya Output
Senior Developer	Code review, arsitektur	Teknis, detail, prioritas
Teacher/Mentor	Explain, tutorial	Analogi, step-by-step, sabar

Role	Konteks	Gaya Output
Technical Writer	Dokumentasi	Rapi, konsisten, formal
Security Auditor	Vulnerability scan	Paranoia, severity rating
Code Reviewer	Pull request review	Constructive, specific

```
// Contoh: Role sebagai Guru
const prompt = `
Kamu adalah guru programming yang jelasin ke anak SMK.
Pake analogi dunia nyata. Target: siswa yang baru belajar 1 semester.

Jelasin konsep closure di JavaScript pake analogi kantin sekolah.
`;
```

Structured Output (JSON Mode)

Paksa AI output dalam format terstruktur biar bisa diparsing program-matically.

Tanpa Structured Output

Buat data produk:

- Nama: Kopi Susu
- Harga: 15000
- Stok: 100
- Kategori: Minuman

AI bisa balik: "Produk Kopi Susu dengan harga Rp15.000 tersedia 100 unit di kategori Minuman" — SUSAH DI-PARSE.

Dengan Structured Output

```
const prompt = `
Buat data produk dalam format JSON. Jangan tambah teks lain.
```

```
{
  "nama": "Kopi Susu",
  "harga": 15000,
  "stok": 100,
  "kategori": "Minuman",
  "tersedia": true
}
```

Sekarang buat produk berikut:

```
Nama: Nasi Goreng, Harga: 25000, Stok: 50, Kategori: Makanan`;
```

```
// Output AI:  
// {  
//   "nama": "Nasi Goreng",  
//   "harga": 25000,  
//   "stok": 50,  
//   "kategori": "Makanan",  
//   "tersedia": true  
// }
```

Prompt Template untuk JSON Mode

```
const jsonPromptSchema = `  
Kamu akan menghasilkan output JSON.  
Schema yang harus diikuti:  
{  
  "field_name": "type | description",  
  "users": "array of objects | list pengguna",  
  "total": "number | total count"  
}
```

```
Hanya output JSON. No markdown. No explanation.  
`;
```

Kegunaan JSON mode di project:

- Generate mock data buat testing
- Parse output AI ke variable program
- Chain prompt (output prompt 1 → input prompt 2)
- Auto-generate configuration files

Iterative Refinement

Prompt pertama hampir never perfect. Refinement = improve prompt bertahap.

Siklus Refinement

1. Tulis prompt awal → eval hasil
2. Identifikasi masalah (vague, wrong format, missing context)
3. Tambah constraint / contoh / spesifikasi
4. Test ulang
5. Repeat sampe acceptable

Contoh Refinement

Iterasi 1 — Terlalu vague:

"Buat fungsi validasi email"

Hasil: fungsi simple, cek @ doang.

Iterasi 2 — Tambah spesifikasi:

"Buat fungsi validasi email di TypeScript.

Validasi: format regex, cek domain valid (gmail.com, yahoo.com), min length 5, max length 100."

Hasil: lebih lengkap, tapi regex terlalu strict.

Iterasi 3 — Tambah constraint:

"Buat fungsi validasi email TypeScript.

- Regex standar RFC 5322 (gampang dibaca, bukan unicode monster)
- Cek domain dari list yang diizinkan: ['gmail.com', 'yahoo.com', 'outlook.com']
- Case insensitive
- Return { valid: boolean, error?: string }
- Unit test juga"

Hasil: sesuai ekspektasi.

// Final result setelah 3 iterasi

```
interface EmailValidationResult {
  valid: boolean;
  error?: string;
}

function validateEmail(email: string): EmailValidationResult {
  const allowedDomains = ['gmail.com', 'yahoo.com', 'outlook.com'];
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

  if (!email || email.length < 5 || email.length > 100) {
    return { valid: false, error: 'Email must be 5-100 characters' };
  }

  if (!emailRegex.test(email)) {
    return { valid: false, error: 'Invalid email format' };
  }

  const domain = email.split('@')[1].toLowerCase();
  if (!allowedDomains.includes(domain)) {
    return { valid: false, error: `Domain ${domain} not allowed` };
  }
}
```

```
    return { valid: true };
}
```

Checklist Evaluasi Prompt

Kriteria	Check
Tujuan jelas?	☐
Format output disebut?	☐
Konteks cukup?	☐
Constraint disebut?	☐
Contoh (few-shot) dikasih?	☐
Role/persona ditentukan?	☐
Bahasa konsisten?	☐

Latihan

1. Zero-Shot vs Few-Shot

Buat 2 prompt untuk task yang sama: klasifikasi kategori produk (Elektronik, Fashion, Makanan, Lainnya).

- Prompt A: zero-shot — tanpa contoh
- Prompt B: few-shot — kasih 3 contoh

Test ke AI. Bandingin hasilnya. Mana yang lebih konsisten?

2. Chain-of-Thought Debugging

Kamu dikasih kode yang error. Tulis prompt CoT buat tracing masalah.

```
function hitungTotal(items: { harga: number; qty: number }[]): number {
    let total = 0;
    for (let i = 0; i <= items.length; i++) {
        total += items[i].harga * items[i].qty;
    }
    return total;
}
```

Prompt harus: suruh AI jelasin tiap baris dulu, baru kasi saran fix. Jangan langsung kasi fix.

3. Role Prompting + JSON Mode

Buat prompt yang pake role "Teknisi Database" dan output dalam format JSON.

Task: generate 3 sample data tabel transactions dengan field:

- id (number)
- user_id (number)
- total (number)
- items (array of { name, qty, price })
- created_at (ISO date)

Output harus pure JSON, no markdown.

4. Iterative Refinement Challenge

Mulai dari prompt ini:

Buat API endpoint untuk login

Refine prompt sampe AI ngasih output yang mencakup:

- TypeScript + Express
- JWT token
- Input validation (email, password min 8)
- Error handling
- Unit test

Dokumentasiin tiap iterasi: apa yang lo tambahkan dan kenapa.

2. Prompt for Code & Production

Durasi: 2 Jam

Fokus: Code generation, test generation, API doc generation, error explanation, code review prompts, Mastra templates, prompt versioning, ethical considerations

Code Generation dengan AI

Code generation = minta AI tulis kode berdasarkan deskripsi. Paling berbahaya kalau lo pake tanpa paham.

Aturan Generate Kode

1. **Minta spesifik** — sebut bahasa, framework, library, pattern

2. **Minta boilerplate doang** — logic bisnis tulis manual
3. **Generate batch kecil** — 1 fungsi per prompt, bukan 1 app
4. **Review setiap baris** — sebelum pake, lo harus paham

// *□ Prompt yang benar – spesifik, terbatas*

```
const prompt = `
Buat Express.js route handler TypeScript untuk GET /api/products.
Gunakan Prisma untuk database.
Return: { success: boolean, data: Product[], message: string }
Handler ini cuma butuh: pagination (page, limit), filter by category.
JANGAN include auth middleware atau logika bisnis lain.
`;
```

// *□ Prompt yang salah – vague, minta full app*

```
const prompt = `
Buat aplikasi toko online pake MERN stack.
`;
```

Contoh Generate Handler

// *Hasil generate dari prompt di atas*

```
import { Request, Response } from 'express';
import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient();

export async function getProducts(req: Request, res: Response) {
  try {
    const page = parseInt(req.query.page as string) || 1;
    const limit = parseInt(req.query.limit as string) || 10;
    const category = req.query.category as string;

    const where = category ? { category } : {};
    const products = await prisma.product.findMany({
      where,
      skip: (page - 1) * limit,
      take: limit,
    });

    res.json({
      success: true,
      data: products,
      message: 'Products fetched successfully',
    });
  } catch (error) {
    res.status(500).json({
```

```

        success: false,
        data: [],
        message: 'Failed to fetch products',
    });
}
}

```

Generate Utility Function

```

const prompt = `
Buat fungsi TypeScript untuk format angka ke format Rupiah.
Contoh: 15000 → "Rp15.000", 1000000 → "Rp1.000.000"
Bisa handle: integer, float, null, undefined, string number.
Unit test juga.
`;

```

// Output AI

```

export function formatRupiah(value: number | string | null | undefined): string {
    if (value === null || value === undefined) return 'Rp0';

    const num = typeof value === 'string' ? parseFloat(value) : value;

    if (isNaN(num)) return 'Rp0';

    return `Rp${Math.round(num)
        .toString()
        .replace(/\B(?=(\d{3})+(?!\d))/g, '.')}`;
}

```

Unit Test Generation

Test case = area di mana AI excel. Pola input → edge case → output sudah jelas.

Prompt Template untuk Generate Test

```

const prompt = `
Buat unit test Jest untuk fungsi di bawah.
Cover:
1. Happy path (positive cases)
2. Edge cases (null, empty, boundary)
3. Error cases (invalid input, exception)

```

Gunakan describe/it pattern. Jangan ubah fungsi asli.

Fungsi yang di-test:
`${fungsiString}`
`;
`;

Contoh Generate Test

```
// Fungsi
export function calculateDiscount(price: number, tier: string): number {
  const discounts: Record<string, number> = {
    silver: 0.05,
    gold: 0.1,
    platinum: 0.2,
  };

  const discount = discounts[tier] || 0;
  return price - price * discount;
}

// AI-generated test
describe('calculateDiscount', () => {
  it('should apply 5% discount for silver tier', () => {
    expect(calculateDiscount(100000, 'silver')).toBe(95000);
  });

  it('should apply 10% discount for gold tier', () => {
    expect(calculateDiscount(100000, 'gold')).toBe(90000);
  });

  it('should apply 20% discount for platinum tier', () => {
    expect(calculateDiscount(100000, 'platinum')).toBe(80000);
  });

  it('should return same price for unknown tier', () => {
    expect(calculateDiscount(100000, 'bronze')).toBe(100000);
  });

  it('should handle zero price', () => {
    expect(calculateDiscount(0, 'gold')).toBe(0);
  });

  it('should handle decimal price', () => {
    expect(calculateDiscount(99999, 'silver')).toBeCloseTo(94999.05);
  });
});
```

```

    it('should handle large numbers', () => {
      expect(calculateDiscount(1_000_000_000, 'platinum')).toBe(800_000_000);
    });
  });

```

Test Coverage Checklist

Skenario	Wajib?
Happy path (input valid)	☐
Invalid input type	☐
Null/undefined	☐
Empty array/string	☐
Boundary values	☐
Large numbers	⚠ Kalau relevan
Concurrent calls	⚠ Kalau async

API Documentation Generation

Minta AI generate JSDoc/TSDoc biar dokumentasi rapi.

```

const prompt = `
Generate TSDoc untuk fungsi di bawah.
Format: @param, @returns, @throws, @example.
Bahasa: Indonesia.

```

```

Fungsi:
`;

```

```

// Contoh fungsi
async function createOrder(
  userId: string,
  items: Array<{ productId: string; qty: number }>,
  shippingAddress: string
): Promise<{ orderId: string; total: number; estimatedDelivery: Date }> {
  // implementation...
}

```

```

// AI-generated TSDoc
/**
 * Membuat pesanan baru untuk user.
 */

```

```

* @param userId - ID user yang memesan
* @param items - Daftar produk yang dipesan (productId + quantity)
* @param shippingAddress - Alamat pengiriman lengkap
* @returns Object berisi orderId, total harga, dan estimasi pengiriman
* @throws {Error} Jika stok tidak mencukupi
* @throws {Error} Jika userId tidak ditemukan
*
* @example
* const order = await createOrder(
*   'user_123',
*   [{ productId: 'prod_456', qty: 2 }],
*   'Jl. Merdeka No. 1, Jakarta'
* );
* // { orderId: 'ORD-001', total: 50000, estimatedDelivery: Date }
*/

```

Error Explanation

Error explanation = minta AI jelaskan error daripada fix langsung.

Prompt Template Error Explain

```
const prompt = `
Saya dapat error di project Node.js + TypeScript.
```

```
Error message:
${errorMessage}
```

```
Stack trace:
${stackTrace}
```

```
Kode yang relevan:
${relevantCode}
```

```
Yang udah saya coba:
- Restart server
- Cek import statement
```

```
Tolong:
1. Jelaskan kenapa error ini terjadi (dalam bahasa Indonesia)
2. Kasih referensi konsep yang harus saya pelajari
3. JANGAN kasih code fix – saya mau coba sendiri
`;
```

Error Explain Flow

Error muncul
↓
Copy error + stack trace + kode relevan
↓
Prompt AI minta penjelasan (bukan fix)
↓
Paham penyebab → fix manual
↓
Kalau mentok → prompt iterasi 2 minta saran

Code Review Prompts

Review kode pake AI sebelum commit. Bedanya dengan "bikin kode":
review fokus evaluasi, bukan generate.

Prompt Review by Scope

```
// Security-focused review
const securityReview = `
Review kode ini dari sisi security only.
Cari:
- SQL injection
- XSS
- CSRF
- Hardcoded secrets
- Insecure direct object reference (IDOR)
- Broken authentication

Prioritas: critical > high > medium > low.
`;

// Performance review
const perfReview = `
Review kode ini dari sisi performa.
Cari:
- N+1 query
- Unnecessary loops
- Memory leak potential
- Sync blocking code di async context
- Large payload tanpa pagination
`;
```

```
// Code quality review
const qualityReview = `
Review kode ini dari sisi maintainability.
Cari:
- Magic numbers
- Duplicate code
- Function terlalu panjang (>50 line)
- Naming yang misleading
- Missing error handling
- Tight coupling
`;
```

Contoh Full Code Review Prompt

```
// Kode yang akan di-review
async function getUsers(req: Request, res: Response) {
  const users = await db.query('SELECT * FROM users');
  let html = '<table>';
  for (let u of users) {
    html += '<tr><td>${u.name}</td><td>${u.email}</td></tr>';
  }
  html += '</table>';
  res.send(html);
}
```

```
// Prompt review
const prompt = `
Review kode ini sebagai senior developer.
Konteks: Express.js API, PostgreSQL.
```

Kode:
\${kodeString}

Evaluasi:

1. Security issues (prioritas tertinggi)
2. SQL injection potential
3. XSS potential
4. Performance bottleneck
5. Code smell
6. Best practice violation

Format output:

Issue	Severity	Line	Explanation
...	Critical/High/Medium/Low	2	...

```
Jangan kasih fix. Cukup identifikasi.  
`;
```

Prompt Templates dengan Mastra

Mastra = framework AI untuk bikin prompt templates terstruktur.

Kenapa Mastra?

- Prompt versioning built-in
- Template variables ({{variable}})
- Multi-provider support (OpenAI, Anthropic)
- Tool calling integration

Mastra Prompt Template Example

```
import { Mastra } from '@mastra/core';  
  
const mastra = new Mastra({  
  apiKey: process.env.OPENAI_API_KEY,  
});  
  
// Define prompt template  
const codeReviewPrompt = mastra.prompt({  
  name: 'code-review',  
  description: 'Review kode untuk security & quality',  
  template: `  
Kamu adalah senior developer yang melakukan code review.  
Konteks proyek: {{projectContext}}  
Teknologi: {{techStack}}  
  
Tugas: review kode berikut dengan fokus pada:  
1. Security vulnerabilities  
2. Performance issues  
3. Code quality & maintainability  
  
Kode:  
\\ \\ \\ \\  
{{codeContent}}  
\\ \\ \\ \\  
`  
`;  
  
Format output:
```

```

    ### Security ({{securityCount}} issues)
    - ...

    ### Performance ({{perfCount}} issues)
    - ...

    ### Quality ({{qualityCount}} issues)
    - ...
  },
});

// Execute template
const review = await mastra.executePrompt('code-review', {
  variables: {
    projectContext: 'Aplikasi POS untuk toko kelontong',
    techStack: 'Next.js 14, Prisma, PostgreSQL',
    codeContent: fs.readFileSync('./src/api/products.ts', 'utf8'),
    securityCount: 3,
    perfCount: 2,
    qualityCount: 3,
  },
});

```

Prompt Template untuk Code Generation

```

const generateHandlerPrompt = mastra.prompt({
  name: 'generate-api-handler',
  description: 'Generate Express/Next.js API handler',
  template: `
Kamu adalah backend developer expert.
Buat {{method}} handler untuk endpoint {{endpoint}}.

Stack: {{stack}}
Database: {{database}}

Spesifikasi:
- Input: {{inputSchema}}
- Output: {{outputSchema}}
- Auth: {{authRequired}}
- Error handling: {{errorHandling}}

### Constraint:
- {{constraint1}}
- {{constraint2}}
- Generated code only, no explanation

```

```
});
```

Version Control untuk Prompts

Prompt = asset yang butuh versioning sama kayak kode.

Kenapa Versioning Penting

- **Rollback** — prompt usang gampang balik
- **Audit trail** — tau siapa ngubah apa dan kapan
- **A/B testing** — bandingin prompt lama vs baru
- **Reproducibility** — hasil lama bisa direproduksi

Strategi Versioning

```
prompts/
├── v1/
│   ├── code-review.md
│   ├── generate-test.md
│   └── error-explain.md
├── v2/
│   ├── code-review.md
│   └── generate-test.md
└── CHANGELOG.md
```

Prompt Changelog Format

Prompts Changelog

v2.0.0 - 2025-03-15

code-review.md

- Added: security severity rating (critical/high/medium/low)
- Changed: output format from prose to markdown table
- Removed: auto-fix suggestion (bikin siswa males)

generate-test.md

- Added: edge case coverage checklist
- Changed: template variable from {{fnName}} to {{functionCode}}

Integrasi dengan Git

Simpan prompt di repo yang sama

```
project/
├── prompts/
│   └── v1/
│       └── code-review.md
├── src/
└── package.json
```

Git tracking

```
git add prompts/
```

```
git commit -m "prompts: update code-review v2 - tambah severity rating"
```

Ethical Considerations dalam Prompt Engineering

❑ Jangan

1. **Prompt injection** — jangan nyuruh AI ngelakuin hal ilegal
2. **Generate malware** — AI bisa, tapi lo jangan
3. **Plagiarism** — generate kode orang lain tanpa atribusi
4. **Over-reliance** — minta AI ngerjain semua tanpa belajar
5. **Data privacy** — jangan tempel credential / API key / data user di prompt

❑ Lakukan

1. **Review output** — setiap baris kode dari AI harus lo paham
2. **Atribusi** — kalau pake prompt orang lain, sebut sumber
3. **Transparansi** — bilang kalau pake AI, dokumentasi di PROMPT-LOG
4. **Limit scope** — AI buat bantu, bukan ganti otak lo
5. **Data sanitasi** — hapus data sensitif sebelum di-prompt

Checklist Etis per Prompt

Pertanyaan	Ya/Tidak
Apaku prompt ini ngasih value ke skill coding lo?	☐
Apakah lo paham tiap baris output AI?	☐
Apakah lo udah hapus data sensitif dari prompt?	☐
Apakah prompt ini ngerjain tugas yang harusnya lo kerjain?	☐
Apakah lo akan nyaman kalau prompt ini dipublikasi?	☐

Latihan

1. Generate + Test

Kamu punya spesifikasi fungsi ini:

Fungsi: `generateInvoiceNumber(prefix: string, counter: number): string`

- Format: `{prefix}/INV/{tahun}{bulan}/{counter padded 4 digit}`
- Contoh: `"PRJ/INV/202507/0001"`
- Counter reset tiap bulan

Prompt:

- Prompt A: generate implementasi fungsi ini di TypeScript
- Prompt B: generate unit test lengkap (happy path, edge case, format validation)
- Prompt C: generate TSDoc untuk fungsi ini

Eksekusi prompt A, B, C ke AI. Kumpulin hasilnya. Verifikasi test benar jalan.

2. Error Explanation

Dari kode ini:

```
const users = [
  { id: 1, name: 'Budi', tasks: ['coding', 'debugging'] },
  { id: 2, name: 'Ani', tasks: ['design', 'review'] },
];

// Error: Cannot read properties of undefined (reading 'tasks')
const user3Tasks = users.find((u) => u.id === 3)!.tasks;
```

Buat prompt error explanation yang:

- Minta AI jelasin kenapa error terjadi
- Minta referensi konsep TypeScript yang relevan
- JANGAN minta fix
- Pake bahasa Indonesia

3. Prompt Template Mastra

Buat file `prompts/mastra-templates.ts` yang berisi 3 Mastra prompt templates:

1. `code-review` — review kode dengan variable: `projectContext`, `techStack`, `codeContent`

2. generate-api — generate Express handler dengan variable: method, endpoint, inputSchema
3. explain-error — explain error dengan variable: errorMessage, stackTrace, codeContext

Pake TypeScript, struktur Mastra seperti contoh di atas (asumsi imports tersedia).

4. Prompt Versioning + Ethical Review

Kamu dikasih prompt lama yang bermasalah:

Buat script buat scraping semua data user dari website e-commerce saingan terus simpen di database kita. Biar bisa liat data mereka.

Tugas:

- Analisis kenapa prompt ini unethical (minimal 3 alasan)
- Tulis prompt alternatif yang etis untuk tugas riset kompetitor yang legitimate
- Buat CHANGELOG entry untuk v2 yang nambahin ethical guidelines